Dental 3D Scanner — Projede Neler Yaptık?

Proje Durum ve Yapılan İşler Kapsamlı Raporu • Emirhan Kar & Furkan Genç • Ekim 2026

Bu projede amacımız; kameradan ve lazer ışığından alınan görüntülerle ağız içindeki dişlerin 3 boyutlu modelini çıkarmak ve yapay zeka kullanarak her bir dişi (FDI standardında 32 diş + diş eti) tek tek otomatik tespit edip birbirinden ayırmaktır (segmentasyon). Sıfırdan kurup çalışır hale getirdiğimiz tüm modüller, derin öğrenme motoru, rekonstrüksiyon boru hattı, test altyapısı ve dağıtım sistemleri aşağıda detaylıca özetlenmiştir:

1. 🧠 Yapay Zeka & Derin Öğrenme Modülü

3D nokta bulutu üzerinden dişleri tanıyan derin öğrenme altyapısının temel taşları tamamlandı:

- **PointNet Tabanlı Segmentasyon Mimarisi (network.py):** Doğrudan 3D noktalar üzerinde çalışan DentalPointNetSeg modelini yazdık. Model her bir nokta için 6 bilgi (X, Y, Z koordinatları ve Nx, Ny, Nz yüzey normalleri) alıyor. Önce yerel diş detaylarını, ardından tüm çenenin global geometrisini çıkarıp birleştiriyor ve her noktayı FDI standardındaki 33 sınıftan (1 diş eti + 32 ayrı diş) birine atıyor.
- **3D Veri Seti Okuyucu ve İşleyici (dataset.py):** Teeth3DS diş veri setindeki yaklaşık 2200 adet 3D diş modelini (.obj) ve bunlara ait diş numarası etiketlerini (.json) otomatik eşleştiren PyTorch veri yükleyicisini kurduk. Mesh'lerden yüzey normalleri hesaplanıyor, etiketler köşe noktalarına bağlanıyor, veriler merkeze alınıp birim küreye ölçekleniyor. Ayrıca modelin ezberlemesini önlemek için rastgele döndürme ve gürültü ekleme (data augmentation) mekanizması yapıldı.
- **GPU Destekli Model Eğitim Döngüsü (train.py):** PyTorch ve CUDA ile ekran kartını kullanan eğitim altyapısı hazırlandı. Veri setini %80 eğitim ve %20 doğrulama olarak bölüyor. Adam algoritması ve CrossEntropy kaybı kullanıldı. Öğrenme oranını belirli aralıklarla azaltan zamanlayıcı (StepLR), en iyi skoru alan modeli diske kaydetme ve eğitim yarım kalırsa kalınan yerden devam edebilme (--resume) özellikleri eklendi.
- **Başarı Ölçüm Metrikleri (metrics.py):** Modelin dişleri ne kadar doğru ayırdığını ölçmek için segmentasyonda standart olan IoU (Kesişim/Birleşim), mIoU (ortalama IoU), Dice katsayısı (F1 skoru) ve nokta doğruluğu (Accuracy) hesaplayan fonksiyonlar yazıldı.
- **İlk Ağırlık Dosyası (best_dental_model.pth):** Model mimarisine uygun şekilde kaydedilmiş yaklaşık 5.3 MB boyutunda bir ağırlık dosyası hazırda bulunuyor ve başarıyla yüklenebiliyor.

2. 📷 Kamera Kalibrasyonu Modülü

- **Kamera Parametre Çıkarıcı (calib_script.py):** Kameranın optik özelliklerini (odak uzaklığı, merkez noktası ve mercek bozulmaları) tespit etmek için satranç tahtası desenli kalibrasyon sistemi kuruldu. OpenCV ile köşeler alt-piksel hassasiyetinde bulunuyor ve kamera matrisi hesaplanıyor.
- **Kalibrasyon Dosyası (camera_params.yaml):** Yapılan hesaplamalar YAML dosyası olarak kaydedildi (1920x1080 çözünürlük, fx=fy=913.66 odak uzaklığı ve 0.1245 piksel gibi oldukça düşük bir yeniden projeksiyon hatası elde edildi).

3. ☸ Merkezi Altyapı ve Yardımcı Araçlar

- **Merkezi Konfigürasyon Sistemi (config.yaml & config.py):** Projeyle ilgili tüm ayarları (kamera, kalibrasyon, yapay zeka hiperparametreleri, dosya yolları, lazer ayarları) tek bir config.yaml dosyasında topladık. Python tarafında cfg.ai_model.batch_size gibi nokta atışı erişim sağlayan bir yapı (ConfigNode) kuruldu.
- **Gelişmiş Loglama Sistemi (logger.py):** Sistemdeki tüm olayları, uyarıları ve hataları hem konsola hem de logs/dental_scanner.log dosyasına kaydeden bir logger yapıldı. Log dosyası 10 MB'a ulaşınca otomatik olarak yedeklenip yeni dosyaya geçiliyor. Windows terminallerinde Türkçe karakterlerin çökme yapması engellendi.
- **Veri ve Model Doğrulayıcı (validator.py):** Veri setindeki veya taramadan çıkan 3D modellerin bozuk, boş veya NaN değerler içerip içermediğini denetleyen; JSON etiketlerinin FDI standartlarına uygunluğunu kontrol eden kapsamlı bir doğrulama ve tarama aracı yazıldı.

4. 🛠 Geliştirme Ortamı ve Temel Düzenlemeler

- **Sanal Ortam ve GPU Desteği:** Bilgisayarda PyTorch CUDA destekli venv_gpu sanal ortamı kuruldu. VS Code editörünün import hataları vermemesi için settings.json dosyası venv_gpu ortamına bağlandı ve proje kökü arama yollarına eklendi.
- **Kod Temizliği ve İsimlendirme:** Kodlar baştan sona gözden geçirildi; kullanılmayan kütüphane importları ve gereksiz döngü değişkenleri temizlendi. Değişkenler ve fonksiyon açıklamaları daha anlaşılır ve Türkçe terimlerle düzenlendi.
- **Bağımsız Doğrulama Testi:** Tüm modüllerin (utils, ai_model, metrics) sorunsuz çalıştığı, modelin hafızaya yüklendiği ve veri setinden gelen bir 3D mesh'i okuyup saniyeler içinde işleyebildiği test edilerek teyit edildi.

5. 🔬 3D Rekonstrüksiyon ve Görüntü İşleme Hattı (processing/)

Donanım sinyallerinden ve kamera karelerinden milimetrik 3B diş modelini adım adım oluşturan fiziksel ve matematiksel rekonstrüksiyon zinciri eksiksiz tamamlandı:

- **Lazer Çizgisi Tespiti ve Alt-Piksel Tepe Analizi (processing/laser_detector.py):** Gelen kamera karelerinde kırmızı/yeşil çizgi lazerini HSV renk uzayında izole eden, gürültüyü morfolojik filtrelerle temizleyen ve çizgi merkezini tam pikselle sınırlamayıp Ağırlık Merkezi (Center of Mass - CoM) ve 1D Gauss eğrisi uydurma formülleriyle alt-piksel (sub-pixel) hassasiyetinde tespit eden modül yazıldı.
- **2D'den 3D'ye Koordinat Dönüşümü ve Üçgenleme (processing/triangulation.py):** Kalibre edilmiş kamera matrisi (K) ile lazer projeksiyon düzlemini kesiştirerek, 2 boyutlu piksel koordinatlarını metrik uzayda gerçek (X, Y, Z) 3B koordinatlarına dönüştüren üçgenleme motoru kuruldu. YAML dosyasından kalibrasyon yükleme ve matris projeksiyon matematiği entegre edildi.
- **Nokta Bulutu Hizalama ve Canlı Dikişleme / ICP (processing/icp_stitcher.py):** Tarayıcı probunun hareketi sırasında peş peşe yakalanan nokta bulutu parçalarını pürüzsüzce birleştirmek için iki aşamalı hizalama mimarisi kuruldu: Kaba eşleme için RANSAC tabanlı FPFH öznitelik analizi, ince dikişleme için diş yüzey normallerini kullanarak kaymayı (drift) sıfırlayan Point-to-Plane ICP algoritması entegre edildi.
- **3B Yüzey Örme ve Katı Model İhracı (processing/mesh_generator.py):** Üretilen milyonlarca noktadan oluşan ayrık nokta bulutlarını diş hekimliği CAD/CAM sistemlerinin ve 3D yazıcıların kullanabileceği kapalı

poligon yüzeylere (.stl, .ply, .obj) dönüştürmek için Poisson Yüzey Rekonstrüksiyonu ve Ball-Pivoting (BPA) yüzey örme algoritmaları entegre edildi.

- **Gelişmiş Nokta Bulutu Filtreleme Hattı (processing/filters.py):** Ağız içi yansımalar ve optik gürültüleri temizlemek için VoxelGrid (hacimsel seyreltme), Statistical Outlier Removal (SOR - istatistiksel aykırı nokta eleme), Radius Outlier Removal (ROR) ve çalışma alanı sınırlayıcı (Crop Box) filtrelerinden oluşan ardışık filtreleme boru hattı kuruldu.

6. 🎯 AI Model Çıkarımı, Canlı Simülasyon ve Arayüz (inference, ui, main)

Geliştirilen yapay zeka modelinin pratikte kullanılmasını sağlayan uçtan uca çıkarım, 3B görselleştirme ve donanımsız test edilebilirlik katmanları inşa edildi:

- **Uçtan Uca 3D Diş Segmentasyon Çıkarımı (ai_model/inference.py):** Eğitilmiş DentalPointNetSeg modelini kullanarak dışarıdan verilen herhangi bir 3D mesh (.obj veya .ply) üzerinde doğrudan çıkarım yapan sistem yazıldı. Her noktayı FDI 32 diş ve diş eti sınıflarına ayırır, her dişe özel kimlik atar ve etiketli PLY/OBJ çıktısı üretir. Hem masaüstü arayüzü hem de sunucu/Docker dostu 'headless' (grafiksiz) mod desteği sağlandı.
- **Etkileşimli 3D Görselleştirme Arayüzü (ui/viewer.py):** Open3D motoru üzerinde çalışan, FDI standartlarına göre 32 dişi ve diş etini birbirinden ayıran 33 farklı renkle kodlanmış etkileşimli 3D sahne arayüzü geliştirildi. Kullanıcının modeli döndürmesine, yakınlaştırmasına ve diş sınırlarını üç boyutta net olarak incelemesine olanak tanır.
- **Asenkron Çok Kanallı Video Akışı (utils/video_stream.py):** Kamera veya video dosyasından kare yakalanırken ağır görüntü işleme ve ICP adımlarının video akışını dondurmasını önlemek için arka planda bağımsız bir iş parçacığında (threading) kare tamponlayan yüksek performanslı asenkron video yakalayıcı kuruldu.
- **Sentetik Simülasyon Modu ve Ana Orkestratör (main.py):** Fiziksel tarayıcı donanımına gerek duymadan tüm boru hattını (lazer tespiti, üçgenleme, filtreleme, ICP dikişleme ve STL ihracı) doğrulamak için parametrik sentetik diş arkı simülasyonu (--simulation) yazıldı. Canlı kamera, video veya simülasyon girdilerini yöneten kapsamlı CLI komut satırı arayüzü tamamlandı.

7. 🧪 Kapsamlı Otomasyon ve Test Altyapısı (tests/ & pytest.ini)

Yazılımın güvenilirliğini, matematiksel doğruluğunu ve platformlar arası stabilitesini teminat altına alan test sistemi oluşturuldu:

- **86 Adet Birim ve Entegrasyon Testi (%100 Başarı):** tests/ dizini altında 14 farklı test dosyasında toplam 86 adet otomatik test geliştirildi. AI tensör boyutları, PointNet forward pass, kalibrasyon matrisleri, lazer alt-piksel analizi, ICP hizalama doğruluğu, Poisson yüzey örme, video akış tamponu ve uçtan uca boru hattı test edildi. 86 testin 86'sı da (%100) başarıyla geçmektedir (86 passed).
- **pytest.ini Standartları:** Test suite'inin standartlaştırılmış bayraklarla, kısa ve anlaşılır hata loglarıyla, zaman aşımı korumasıyla hem geliştirici bilgisayarında hem de CI/CD ve Docker üzerinde tek komutla koşturulması sağlandı.

8. 🐳 Docker Konteynerizasyonu & Çoklu Servis Mimarisi

Sistemi sıfır ortam kurulumuyla herhangi bir sunucuda veya bilgisayarda çalıştırmayı sağlayan Docker altyapısı hazırlandı:

- **Çok Katmanlı Dockerfile:** Ubuntu tabanlı; Python, PyTorch, CUDA, OpenCV, Open3D ve gerekli grafik bağımlılıklarını izole bir şekilde barındıran üretim kalitesinde Docker imajı tanımlandı.
- **Modüler Docker Compose Servisleri (docker-compose.yml):** Farklı kullanım senaryoları için tek komutla ayağa kaldırılabilen 4 bağımsız servis yapılandırıldı:
  • app: Sentetik simülasyonu çalıştırıp çıktı üreten servis
  • test: 86 birim ve entegrasyon testini izole ortamda koşturan servis
  • ai-train: Derin öğrenme model eğitimini başlatan servis
  • ai-infer: 3D diş segmentasyonu çıkarımını yürüten servis
- **Derleme Optimizasyonu (.dockerignore):** Büyük boyutlu veri setlerinin, yerel sanal ortamların ve önbelleklerin konteyner içine girmesini engelleyerek hızlı derleme ve minimum imaj boyutu elde edildi.

9. 📦 Veri Seti Dağıtımı, Lisanslama ve Dokümantasyon

- **16.25 GB Teeth3DS Veri Seti Bulut Entegrasyonu:** Büyük boyutlu 3D diş tarama modelleri ve etiketleri (Dental_3DTeeth_Dataset.zip) için doğrudan indirilebilir Google Drive bulut paylaşım altyapısı kuruldu ve proje README dokümanına entegre edildi.
- **Kapsamlı Teknik Dokümantasyon (README.md):** Veri akış diyagramı, alt-piksel ağırlık merkezi formülü, üçgenleme projeksiyon matematiği, Point-to-Plane ICP optimizasyon prensipleri, dizin ağacı, Docker ve CLI kullanım kılavuzlarıyla donatılmış profesyonel dokümantasyon oluşturuldu.
- **Resmileştirilmiş MIT Lisansı (LICENSE):** Proje telif hakları Emirhan Kar ve Furkan Genç ortak mülkiyeti olarak MIT lisansıyla resmileştirildi.

📊 Güncel Modül ve Bileşen Durum Tablosu

Modül / Bileşen	İlgili Dosyalar & Konum	Mevcut Durum
3D AI Modeli & Eğitim	ai_model/ (network, dataset, train, metrics)	**Tamamlandı, test edildi ve aktif**
3D AI Çıkarım Motoru	ai_model/inference.py	**Tamamlandı, GUI & Headless destekli**
Lazer & Alt-Piksel Tespiti	processing/laser_detector.py	**Tamamlandı, CoM ve Gaussian aktif**
2D→3D Üçgenleme Motoru	processing/triangulation.py	**Tamamlandı, K matrisi entegre**
Nokta Bulutu ICP Dikişleme	processing/icp_stitcher.py	**Tamamlandı, Point-to-Plane + RANSAC**
3B Yüzey Örme (Mesh)	processing/mesh_generator.py	**Tamamlandı, Poisson & BPA aktif**
Nokta Bulutu Filtreleri	processing/filters.py	**Tamamlandı, Voxel, SOR, ROR aktif**
Kamera Kalibrasyonu	calibration/ (calib_script, camera_params.yaml)	**Tamamlandı, 0.12 px hata oranı**
İnteraktif 3D Görselleştirici	ui/viewer.py	**Tamamlandı, FDI 32 diş renklendirmeli**
Asenkron Video Akışı	utils/video_stream.py	**Tamamlandı, çoklu iş parçacığı tamponu**
Sentetik Simülasyon & CLI	main.py	**Tamamlandı, uçtan uca pipeline orkestratörü**
Ayar, Log & Doğrulama	configs/config.yaml, utils/logger, validator	**Tamamlandı, merkezi altyapı devrede**
Otomatik Test Altyapısı	tests/ (14 dosya), pytest.ini	**Tamamlandı (86 testin 86'sı geçti)**

Docker Konteynerizasyonu	Dockerfile, docker-compose.yml, .dockerignore	**Tamamlandı (4 bağımsız servis)**
Veri Dağıtımı & Dokümantasyon	data/ (16.25 GB Drive bağlantısı), README.md	**Tamamlandı, lisanslı ve yayında**